Week 2b: Probability

Nadav Kohen

June 29, 2025

In Week 2a we learned to phrase "problem X is hard" as the statement that every program has negligible *advantage* in an attack game, and we learned to relate one hardness assumption to another using proof by reduction. We also saw proof by reduction used to prove that some security claim followed from a hardness assumption. Throughout, the advantage was written in terms of a probability function $\Pr[\cdot]$ that we treated as a black box. In this second part of the week we open that black box. We will develop just enough probability theory to define $\Pr[\cdot]$ carefully, and then we will put it to work: we will give a precise security definition for symmetric-key encryption and use probability, together with the reduction technique from Week 2a, to prove our first real security theorems. Because all of the sets we care about in this course (keys, messages, ciphertexts) are finite, we will be able to get away with a very small amount of theory.

It will help to have a concrete goal to aim the theory at, so we begin by setting up the security question we want to answer, and only then build the tools to answer it.

A motivating problem: when is an encryption scheme secure?

Let us pivot to the context of symmetric-key encryption, which lets us study what security definitions look like in a simpler setting without public keys, so that we can prove some substantial facts. (We will return to the public-key cryptographic assumptions from Week 2a when we study Schnorr signatures next week.)

Recall from Reading 4 of Week 1 that one example of a symmetric cipher is the one-time pad, where our keys, messages, and ciphertexts are all λ-bit binary strings, our encryption function is $E(k, m) = k \oplus m$ and our decryption function is $D(k, c) = k \oplus c$, where $\oplus$ is bit-wise xor. We want to precisely make the claim that the one-time pad is a secure encryption function, but what does this mean? One phrasing we might try is that for any two messages, m_1 and m_2, no adversary (who does not know k) can distinguish whether a ciphertext, c, corresponds to an encryption of m_1 or an encryption of m_2. But this leaves a few questions unanswered: What is an adversary? Which messages are being used? Who chooses the messages and who encrypts them?

As with the cryptographic assumptions in Week 2a, in security definitions we make these kinds of statements precise and functional by formalizing them as attack games. In this case, we have to define a game where the adversary is challenged to distinguish what message a ciphertext corresponds to. In order to have the strongest possible notion of security, we give the adversary as much information and control as we can in the game, so we will let the adversary pick the messages m_1 and m_2, the adversary will then give these messages to the challenger and the challenger will generate a random key, k, choose a random message to encrypt, and return this encryption to the adversary. Finally, the adversary will have to guess whether the ciphertext it was given is an encryption of m_1 or of m_2. The adversary

wins this game if its guess is correct, and loses otherwise. A succinct definition of this game is given by the $\mathsf{DistinguishCipher}^{\mathcal{A}}(\lambda)$ program in Figure 1.

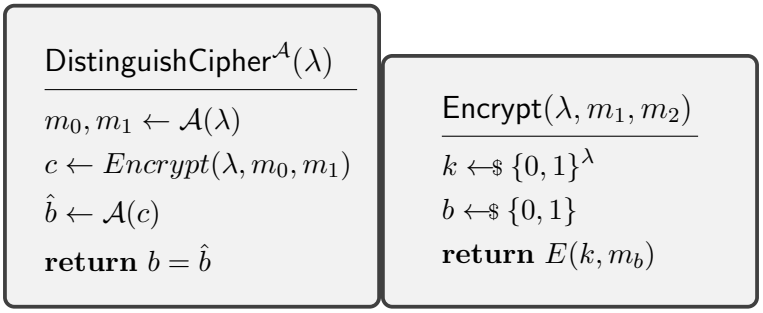

Figure 1: Encryption Distinguishing game ($ means chosen uniformly at random)

Since an adversary that just picks all of its values randomly wins the $\mathsf{DistinguishCipher}^{\mathcal{A}}(\lambda)$ game half of the time, we define the advantage an adversary, $\mathcal{A}$, has in this game to be

$$\mathsf{Adv}^{\text{distinguishcipher}}_{\mathcal{A}}(\lambda) := \Pr\left[\mathsf{DistinguishCipher}^{\mathcal{A}}(\lambda) = \text{true}\right] - \frac{1}{2}.$$

Finally, we can define E to be secure if for all adversary programs $\mathcal{A}$, the advantage of $\mathcal{A}$ is negligible.

This is a perfectly precise definition — except that, just like in Week 2a, it leans entirely on the probability function $\Pr[\cdot]$, and to actually *prove* anything about a scheme we will have to compute quantities like $\Pr\left[\mathsf{DistinguishCipher}^{\mathcal{A}}(\lambda) = \text{true}\right]$. So let us now develop the probability we need, and then we will come back and prove that the one-time pad is secure.

The basics of (finite) probability

In order to work with security definitions such as the one above, we will need to develop a little bit of background in probability so that we can properly reason about advantages. All of our sets (for example, of possible keys or of possible messages) are finite, so we will not need to introduce very much of the general theory. Whenever discussing probabilities

there will implicitly be a (finite) set of possible states/outcomes, Ω, and subsets of this state space are known as events. For example, if we are rolling a 6-sided die to set our (very weak) private key, we could model this situation with $\Omega = \{1, 2, 3, 4, 5, 6\}$ and the event that you roll an even number (i.e., the event that the least significant bit is 0) is given by the subset $\{2, 4, 6\}$. For our purposes, given a state space, Ω, the function $\Pr[E]$ that takes events as input is defined as follows:

1. $\Pr[E] \geq 0$ for all events $E \subseteq \Omega$. That is, probability is not negative.
2. $\Pr[\Omega] = 1$. That is, the probability that one of our outcomes occurs is 1.
3. $\Pr[E] = \sum_{x \in E} \Pr[\{x\}]$. In other words, every individual state has a probability and the probability of an event is the sum of the probabilities of its elements.

Thus, if we assume our 6-sided die has each of its outcomes occurring with probability $\frac{1}{6}$ (which satisfies our probability rules), then the probability of rolling an even number is $\frac{1}{6} + \frac{1}{6} + \frac{1}{6} = \frac{3}{6} = \frac{1}{2}$, as we would expect. In general, we call the assignment of probabilities to individual states a *probability distribution* (assuming all the values are non-negative and add up to 1). For example, if we have a set of N outcomes, Ω, then the *uniform* probability distribution on Ω is the assignment of $\Pr[x] = \frac{1}{N}$ for every $x \in \Omega$; when working with uniform distributions, computing probabilities is always the same as counting the number of states in an event (and then dividing by N).

With all of this in mind, when our game says $k \xleftarrow{\$} \{0,1\}^\lambda$, it is declaring that k is drawn from the uniform distribution on the 2^λ possible keys, so each particular key has probability $\frac{1}{2^\lambda}$. Computing the probability of some event about k will, for us, almost always come down to counting.

Exercise 1. *Warm-up. A single key k is drawn uniformly at random from $\{0,1\}^3$ (so Ω has 8 states, each of probability $\frac{1}{8}$). Using only the three rules above, compute: (a) the*

probability that k starts with a 0; (b) the probability that k has an even number of 1s; (c) the probability that $k = 010$. For each, write down the event as a subset of Ω before computing its probability, so that you are explicitly using rule 3.

Exercise 2. *Suppose there is a room with 30 people, and assume that the probabilities of all assignments of one of the 365 possible birthdays to each of the 30 people are equal (this is of course not true, but assuming a uniform distribution will make things simpler). Compute the probability that no two people share a birthday, and then compute the probability that at least two people share a birthday.*
(If instead of people and birthdays we think of inputs and hash values, this "birthday problem" shows that finding hash collisions is easier than finding hash pre-images).

If you would like an additional resource on probability that moves through some of the above topics in a different way and with more depth (and rigor) with more examples, see the following optional reading. However unless you're full of free time, I strongly recommend not spending your time on this unless you are actually getting badly stuck, and even then maybe consult me or discourse for help first.

Reading 1 (Optional)**.** *If you find yourself wanting more probability practice, read about the axioms of probability and conditional probability in:*
Ross - First Course in Probability (5th ed.) - pages 25-39, 67-89.

Exercise 3. *Explain why the following is true using the definition of probability:*

$$Pr\left[\bigcup_{i=1}^{n} E_i\right] \leq \sum_{i=1}^{n} Pr\left[E_i\right].$$

When are these two values equal? (Hint: first try the case where $n = 2$).

In case this notation is not familiar to you, $\bigcup_{i=1}^{n} E_i = E_1 \cup E_2 \cup \cdots \cup E_n$ refers to the union of all of the sets E_1 through E_n, and $\sum_{i=1}^{n} Pr[E_i] = Pr[E_1] + Pr[E_2] + \cdots + Pr[E_n]$

refers to the sum of all of the probabilities $Pr[E_1]$ through $Pr[E_n]$. You can think of both of these as essentially for loops.
(This inequality is called the union bound*, and it is one of the most-used tools in all of cryptography: it lets us upper-bound the probability that* anything *goes wrong by the sum of the probabilities of each individual bad event.)*

Conditional probability and independence

An important notion in probability, that we have already used informally in Week 2a, is when two events are *independent*. Intuitively this means that two events are uncorrelated. In order to capture this notion precisely, we first introduce the notion of *conditional probability*:

$$\Pr[E_1 \mid E_2] := \frac{\Pr[E_1 \cap E_2]}{\Pr[E_2]}.$$

This formula is read "the probability that E_1 occurs *given* that E_2 occurs is equal to the probability that E_1 and E_2 both occur divided by the probability that E_2 occurs." This makes sense in our definition of probability because the information that E_2 occurs essentially is the same as replacing our original set of possible outcomes, Ω, with E_2 instead, and re-interpreting the event E_1 as the set of states in E_1 that are also in our new set of possible outcomes, E_2, throwing away the rest of the states. Note that this definition requires that $\Pr[E_2] \neq 0$ to avoid division by 0, but this is of no consequence since there isn't much use in conditionalizing on a probability 0 event.

An important and immediate result is that $\Pr[E_1 \cap E_2] = \Pr[E_1 \mid E_2] \cdot \Pr[E_2]$. This observation allows us to compute probabilities by breaking a state space up into cases. Suppose that the state space has a partition into two pieces: $\Omega = A \cup B$ and there is no intersection between these pieces ($A \cap B = \emptyset$). Then we can compute the probability of any

event, E, by splitting into the A and B cases using rule 3 as follows,

$$\begin{aligned}\Pr[E] &= \Pr[E \cap A] + \Pr[E \cap B] \\ &= \Pr[E \mid A] \cdot \Pr[A] + \Pr[E \mid B] \cdot \Pr[B].\end{aligned}$$

And more generally, it similarly follows that for any partition $\Omega = \cup_{i=1}^{n} A_i$ (such that none of the A_i have any states in common), we have

$$\Pr[E] = \sum_{i=1}^{n} \Pr[E \cap A_i] = \sum_{i=1}^{n} \Pr[E \mid A_i] \cdot \Pr[A_i].$$

One way to think about this equation (called the *law of total probability*) is that it says that you can always compute a probability, $\Pr[E]$, as a weighted average of the conditional probabilities that E occurs over some partition of the state space, where the weights used are the probabilities of the partition elements.

Now that we have conditional probabilities, we can define what it means for two events to be independent: We say that E_1 and E_2 are *independent events* if $\Pr[E_1 \mid E_2] = \Pr[E_1]$. Intuitively, this says that learning E_2 happened tells you nothing about whether E_1 happened.

Exercise 4. *Warm-up for the law of total probability. A fair coin is flipped. If it comes up heads, you roll a* 6*-sided die; if it comes up tails, you roll a* 4*-sided die (with faces* $1, 2, 3, 4$*). Let* E *be the event that the die shows a* 3*. Define* Ω *for this experiment and use the partition of the outcome space into the "heads" and "tails" events to compute* $Pr[E]$ *as* $Pr[E \mid heads] \cdot Pr[heads] + Pr[E \mid tails] \cdot Pr[tails]$*. This is exactly the case-splitting move you will need in the one-time pad proof below.*

Exercise 5. *Using the definitions above, show that two events,* E_1 *and* E_2*, are independent if and only if* $P(E_1 \cap E_2) = P(E_1) \cdot P(E_2)$*. This is the usual textbook definition of independent*

events, but it sometimes hides the intuition behind what this has to do with independence.

Proving the one-time pad secure

Exercise 6 (Simple but hard)**.** *We now have all of the tools we need to prove that the one-time pad is secure!*

(a) Let us begin with the example of the single-bit encryption case (where $\lambda = 1$). Show that for all adversary programs, $\mathcal{A}$, it is always true that $\mathsf{Adv}_{\mathcal{A}}^{\text{distinguishcipher}}(\lambda) = 0$.

(Hint: let $\Omega = \mathcal{K} \times \{0,1\} \times R$ be the set of triples with one element from $\mathcal{K} = \{0,1\}$, another bit element from $\{0,1\}$, and an element of some set, R, which represents the randomness used by our adversary (these are sources of randomness). Show that if m_0 and m_1 are any pair of messages, then $Pr[\mathcal{A}(c) = b] = \frac{1}{2}$, where $c = E(k, m_b) = k \oplus m_b$ and $\mathcal{A}$ is any program, by breaking into the four cases $(k, b) = (0,0), (0,1), (1,0), (1,1)$ and then using the partition of Ω into sets where k and b are fixed to compute $Pr[\mathcal{A}(c) = b]$. Note that you will also have to treat separately the case that $m_0 = m_1$ and the case that $m_0 \neq m_1$).

(Extra hint if you get stuck near the end: Remember $Pr[\mathcal{A}(c) = 0] + Pr[\mathcal{A}(c) = 1] = 1$ for any fixed value of c since the probability that $\mathcal{A}$ returns an output must be 1).

(b) Using the same general argument as in the previous part, show that $\mathsf{Adv}_{\mathcal{A}}^{\text{distinguishcipher}}(\lambda) = 0$ for all λ and for all adversaries, $\mathcal{A}$.

The general idea here is that since $k \oplus m$ is independent of m when k is uniformly random, no program can use $k \oplus m$ as input to gain any information at all. This is actually much stronger than our requirement for security; having $\mathsf{Adv}_{\mathcal{A}}^{\text{distinguishcipher}}(\lambda) = 0$ instead of just having it be negligible, is called *perfect secrecy*, which was a notion developed by Shannon as he was inventing information theory. Unfortunately, Shannon also proved a theorem showing that if we require perfect secrecy, then our key sizes must always be at

least as large as message sizes (that is, $|\mathcal{K}| \geq |\mathcal{M}|$). This is in stark contrast to the world we are used to, where fixed key sizes can be used to encrypt very large messages. This is often accomplished by stretching a key by using it as the seed to a pseudo-random generator and then using the output of this pseudo-random generator to do encryption. Our final goal this week is to put together much of what we have learned so far to prove that this is secure! This accomplishment demonstrates the power of using negligible advantages instead of advantages equal to 0 in security definitions.

Stretching a key: pseudo-random generators and stream ciphers

A *pseudo-random generator* (PRG), G, is an efficient deterministic algorithm that takes as input an input from $\mathcal{S} = \{0,1\}^\ell$ and outputs an element from $\mathcal{R} = \{0,1\}^L$ where $L > \ell$ (for our purposes this week, we can take $L = \ell + C$ for some large constant C). We often call the elements of $\mathcal{S}$ seeds.

Exercise 7. *Using the security definitions we have seen so far as a model, define an attack game that calls on an adversary to distinguish between an output of G and a truly random element. Define the advantage an adversary has in this game, and define what it means for a PRG, G, to be secure in terms of advantages.*

We define the *stream cipher* associated to a PRG, G, to be the functions (E_G, D_G) given by $E_G(k,m) = G(k) \oplus m$ and $D_G(k,c) = G(k) \oplus c$, which is like the one-time pad except that we first stretch our key using G.

Exercise 8. *Using the definition of a secure cipher given in Figure 1, and your definition of a secure PRG from the previous exercise, use proof by reduction (the technique from Week 2a) to show that if G is a secure PRG, then E_G is a secure encryption function. You may*

use the fact, that we will prove next week, that if $F(\lambda)$ is not negligible, then $\frac{F(\lambda)}{2}$ is also not negligible.

Hint: Given a program $\mathcal{A}$ that has a non-negligible advantage against the game in Figure 1 for E_G, construct an adversary program $A'(L, r)$ that plays your game from the previous exercise by using r as a one-time pad key to encrypt a random message given by $A(L)$. To argue that A' has a non-negligible advantage, use the law of total probability to split on the challenger's hidden bit:

$$Pr[\mathcal{A}' \text{ wins}] = \frac{1}{2} \cdot Pr[\mathcal{A}' \text{ wins} \mid r \text{ is random}] + \frac{1}{2} \cdot Pr[\mathcal{A}' \text{ wins} \mid r = G(s)] \,.$$

Notice that this is the same case-splitting move you practiced in Exercise 4, now doing real work: one term is governed by the perfect secrecy of the one-time pad, and the other by $\mathcal{A}$'s advantage against E_G.